

PHP CRUD II

José Manuel Sánchez Álvarez

Contents

1	CRUD Refactoring	1
1.1	PHP include, include_once, require, require_once	1
1.2	Refactoring CRUD Application Using Includes	2
1.2.1	Example Structure	2
1.2.2	Implementing Includes in CRUD Scripts	3
1.3	Benefits of Using Includes	3
1.4	Best Practices	3

1 CRUD Refactoring

Creating a PHP CRUD application often involves repetitive code, especially for database connections and HTML structures. Using PHP's `include`, `include_once`, `require`, and `require_once` statements can streamline this process by allowing you to write common code in separate files and include them where needed. Here's a guide on how to use these statements effectively, along with tips for refactoring a CRUD application.

1.1 PHP include, include_once, require, require_once

1. include:

- Used to include and evaluate a specified file.
- If the file is not found, PHP issues a warning, but the script will continue execution.

```
<?=  
include 'filename.php';
```

2. include_once:

- Similar to `include`, but it will only include the file once, even if the statement appears multiple times.
- Useful to prevent re-declaring functions, classes, or variable values.

```
<?=  
include_once 'filename.php';
```

3. require:

- Similar to `include`, but `require` will produce a fatal error (halting script execution) if the file cannot be included.
- Use `require` when the file is essential for the application to run.

```
<?=
require 'filename.php';
```

4. `require_once`:

- Combines `require` and `include_once` behavior.
- Ensures that the file is included only once and execution stops if the file can't be included.

```
<?=
require_once 'filename.php';
```

1.2 Refactoring CRUD Application Using Includes

When refactoring your CRUD application, identify parts of your code that are common across multiple scripts. Typically, these include:

- Database Connection Setup
- Header/Footer HTML Code
- Utility Functions

1.2.1 Example Structure

1. Database Connection (`db_connect.php`):

```
<?=
// db_connect.php
$host = 'localhost';
$db    = 'car_dealership';
$user  = 'username';
$pass  = 'password';
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
try {
    $pdo = new PDO($dsn, $user, $pass);
    // Set error mode
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (\PDOException $e) {
    throw new \PDOException($e->getMessage(), (int)$e->getCode());
}
```

2. Header/Footer HTML (`header.php` and `footer.php`):

```
// header.php
```

```

<!DOCTYPE html>
<html>
<head>
    <title>Car Dealership</title>
    <!-- Add other head elements -->
</head>
<body>

// footer.php
    <!-- Footer content -->
</body>
</html>

```

3. Utility Functions (utilities.php):

```

<?php
// utilities.php
function sanitizeInput($data) {
    return htmlspecialchars(stripslashes(trim($data)));
}

```

1.2.2 Implementing Includes in CRUD Scripts

In your CRUD scripts, you can now include these files to avoid redundancy. For example, in a script where you list cars (list_cars.php):

```

<?php
require_once 'db_connect.php';
include 'header.php';
// ... Rest of your code to list cars ...
include 'footer.php';

```

1.3 Benefits of Using Includes

- **Maintainability:** Changes in common elements like database configuration or HTML structure can be made in one place.
- **Readability:** Reduces clutter in your scripts, making them easier to read and manage.
- **Reusability:** Commonly used code can be reused across different scripts.

1.4 Best Practices

- Use `require_once` for critical files like database connections, where the script should not proceed if the file is missing.
- Use `include_once` for elements like headers and footers where the inclusion is important but not critical.
- Regularly review your scripts to identify opportunities for modularizing your code with includes.

By following this guide, you can refactor your PHP CRUD application to be more efficient, maintainable, and scalable.